

Document 4 — Website Setup Instruction Manual

SIH26166 · Frontend, backend, Supabase, Upstash, Sentry, Google AI Studio —
click-by-click

Drafted August 31, 2026

Contents

1. Prerequisites	2
2. Supabase — backend database, auth, storage	2
3. FastAPI compute backend	4
4. Upstash Redis	6
5. Sentry	6
6. Google AI Studio (Gemini API)	8
7. Frontend — React + Vite + React Three Fiber	8
8. Deployment	9
9. Local end-to-end smoke test, before demo day	9

1. Prerequisites

- Node.js 20+, Python 3.11+, git, a GitHub account (for deploy hooks).
- Accounts: Supabase, Upstash, Sentry, Google AI Studio (Gemini API). All have free tiers sufficient for a hackathon build.

2. Supabase — backend database, auth, storage

1. Go to supabase.com → New project. Choose a region close to your judging venue if latency matters for the live demo.
2. Once provisioned, go to Project Settings → API. Copy:
 - Project URL → this is your SUPABASE_URL
 - anon public key → SUPABASE_ANON_KEY (safe for frontend)
 - service_role key → SUPABASE_SERVICE_ROLE_KEY (backend only, never ship this to the browser)
3. Go to SQL Editor and run:

```
create table scenes (  
  id uuid primary key default gen_random_uuid(),  
  instrument text not null,  
  source_path text,  
  reference_path text,  
  footprint jsonb,  
  created_at timestamptz default now()  
);  
  
create table jobs (  
  id uuid primary key default gen_random_uuid(),  
  scene_id uuid references scenes(id),  
  stage text,  
  status text default 'pending',  
  created_at timestamptz default now(),  
  updated_at timestamptz default now()  
);  
  
create table matches (  
  id uuid primary key default gen_random_uuid(),  
  job_id uuid references jobs(id),  
  keypoints_source jsonb,  
  keypoints_ref jsonb,  
  homography jsonb  
);  
  
create table metrics (  
  id uuid primary key default gen_random_uuid(),  
  job_id uuid references jobs(id),  
  rmse numeric,  
  inlier_count int,  
  inlier_ratio numeric,  
  method text  
);
```

4. Go to Storage → New bucket, create raw-tiles, dem-patches, model-weights. Set them private (not public) since PRADAN data likely has usage terms.
5. (Optional, multi-user) Go to Authentication → Providers, enable email/password or GitHub OAuth; add Row-Level Security policies scoping rows to `auth.uid()` if more than one team account will write data.

3. FastAPI compute backend

```
mkdir lunar-registration && cd lunar-registration
python3 -m venv .venv && source .venv/bin/activate
pip install fastapi uvicorn python-dotenv supabase onnxruntime-gpu \
    opencv-python rasterio numpy scipy scikit-image \
    upstash-redis sentry-sdk google-generativeai
```

main.py skeleton:

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
import sentry_sdk, os
from dotenv import load_dotenv

load_dotenv()
sentry_sdk.init(dsn=os.environ["SENTRY_DSN_BACKEND"], traces_sample_rate=0.2)

app = FastAPI()
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], allow_methods=["*"], allow_headers=["*"],
)

@app.post("/ingest")
async def ingest(payload: dict): ...

@app.post("/geometry")
async def geometry(payload: dict): ...

@app.post("/correct")
async def correct(payload: dict): ...

@app.post("/match")
async def match(payload: dict): ...

@app.post("/verify")
async def verify(payload: dict): ...

@app.get("/evaluate/{job_id}")
async def evaluate(job_id: str): ...

@app.get("/visualize/{job_id}")
async def visualize(job_id: str): ...
```

Run locally: `uvicorn main:app --reload --port 8000`.

.env file (backend, never committed — add to .gitignore):

```
SUPABASE_URL=...
SUPABASE_SERVICE_ROLE_KEY=...
UPSTASH_REDIS_REST_URL=...
UPSTASH_REDIS_REST_TOKEN=...
SENTRY_DSN_BACKEND=...
```

```
GOOGLE_API_KEY=...
```

4. Upstash Redis

1. Go to upstash.com → Create database → choose Regional (cheapest/simplest for a hackathon; Global only if you need multi-region low latency, which you don't here).
2. From the database dashboard, copy UPSTASH_REDIS_REST_URL and UPSTASH_REDIS_REST_TOKEN.
3. Backend usage (Python, REST client):

```
from upstash_redis import Redis
redis = Redis(
    url=os.environ["UPSTASH_REDIS_REST_URL"],
    token=os.environ["UPSTASH_REDIS_REST_TOKEN"],
)

def set_job_status(job_id: str, status: str):
    redis.set(f"job:{job_id}:status", status, ex=3600)

def get_job_status(job_id: str):
    return redis.get(f"job:{job_id}:status")
```

4. Rate-limit the Gemini narration endpoint with a simple sliding counter:

```
def allow_narration_call(user_key: str, limit: int = 20, window: int = 60):
    count = redis.incr(f"rl:{user_key}")
    if count == 1:
        redis.expire(f"rl:{user_key}", window)
    return count <= limit
```

5. Sentry

1. Go to sentry.io → Create project → choose Python (FastAPI) for one project, React for a second.
2. Backend: pip install "sentry-sdk[fastapi]", then the sentry_sdk.init(...) call shown in §3 above, using the DSN from the FastAPI project.
3. Frontend:

```
npm install @sentry/react
```

```
// main.jsx
import * as Sentry from "@sentry/react";
Sentry.init({
    dsn: import.meta.env.VITE_SENTRY_DSN_FRONTEND,
    integrations: [Sentry.browserTracingIntegration()],
    tracesSampleRate: 0.2,
});
```

4. Add checkpoint breadcrumbs after each pipeline stage in the backend so a silently-wrong band-order read or coordinate-transform bug surfaces as a Sentry event instead of a silent bad result:

```
sentry_sdk.add_breadcrumb(
    category="pipeline",
```

```
message=f"Stage 4 correction complete for job {job_id}",  
)
```

6. Google AI Studio (Gemini API)

1. Go to aistudio.google.com → Get API key → create a key, copy it into `GOOGLE_API_KEY`.
2. Before hard-coding a model string, check ai.google.dev/gemini-api/docs/models for the current lineup — this list changes roughly monthly. At time of writing it includes `gemini-3.7-flash` (latest, agentic-tuned) and `gemini-3.1-pro` (strongest reasoning); verify before shipping.
3. Backend usage (narration only — never for generating metrics):

```
import google.generativeai as genai
genai.configure(api_key=os.environ["GOOGLE_API_KEY"])
model = genai.GenerativeModel("gemini-3.7-flash") # confirm current string at
↳ ai.google.dev before demo day

def narrate(metrics: dict) -> str:
    prompt = (
        f"Explain in one paragraph, for a non-technical judge, "
        f"why RMSE changed as shown: {metrics}"
    )
    return model.generate_content(prompt).text
```

7. Frontend — React + Vite + React Three Fiber

```
npm create vite@latest lunar-frontend -- --template react
cd lunar-frontend
npm install @supabase/supabase-js @react-three/fiber @react-three/drei three
↳ @sentry/react
```

`.env` (frontend, `VITE_` prefix required by Vite):

```
VITE_SUPABASE_URL=...
VITE_SUPABASE_ANON_KEY=...
VITE_SENTRY_DSN_FRONTEND=...
VITE_API_BASE_URL=http://localhost:8000
```

`src/supabaseClient.js`:

```
import { createClient } from "@supabase/supabase-js";
export const supabase = createClient(
    import.meta.env.VITE_SUPABASE_URL,
    import.meta.env.VITE_SUPABASE_ANON_KEY
);
```

Run: `npm run dev`.

8. Deployment

- **Frontend:** push to GitHub, import into Vercel or Netlify, set the `VITE_*` environment variables in the dashboard, deploy. Both auto-deploy on push, which is useful right up until demo day.
- **Backend:** for a hackathon timeline, the simplest reliable option is a small always-on VM (a \$5–10/month droplet on DigitalOcean/Render/Railway) running `uvicorn main:app --host 0.0.0.0 --port 8000` behind a reverse proxy, or a managed platform like Render/Railway’s native FastAPI deploy (push-to-deploy, no server management). Avoid serverless for the compute backend specifically — cold starts and execution-time limits are a bad fit for GPU-adjacent inference calls.
- **CORS:** once you have a real frontend origin, tighten `allow_origins` in `main.py` from `["*"]` to your actual deployed URL before the live demo.

9. Local end-to-end smoke test, before demo day

1. `uvicorn main:app --reload` (backend running).
2. `npm run dev` (frontend running).
3. Upload/select a scene in the UI → confirm a row appears in Supabase’s scenes table.
4. Trigger `/match` → confirm a Redis key `job:{id}:status` updates and a matches row is written.
5. Force an error (e.g. malformed payload) → confirm it shows up in both Sentry projects, not just a console log.
6. Call the narration panel → confirm a Gemini response renders, and confirm the Upstash rate limiter actually blocks a 21st rapid call in the same minute.

If all six of those pass, the wiring described in this document is genuinely working end to end.